

Niveau : II1 Date : Durée : 2 heures Nombre de pages : 3	Documents : Non autorisés Calculatrice : Non Autorisée Annexe : Non Barème à titre indicatif : (10+10)	 Ecole Nationale des Sciences de l'Informatique Université de la Manouba A.U. 2024/2025
Enseignantes : D. DHOUIB, I. AMDOUNI, N. DRIDI, M. BEN SASSI, R. CHEBIL		Réf : ID.104 Version :01
EXAMEN DE LA SESSION DE RATTRAPAGE Algorithmique et Structures de Données		

Exercice 1 (10 points)

On s'intéresse dans cet exercice à la boîte de messages d'un smartphone et on souhaite développer des opérations permettant sa gestion automatique. Puisque l'ordre des messages dans la boîte de réception suit le principe **Last In First Out**, celle-ci sera représentée par le TAD « **Pile** ». Comme illustré par la figure 1, chaque message est représenté par : le numéro de l'émetteur (exp : +21620123456), l'identifiant unique du message (entier), la priorité (1 ou 2) et le contenu du message.

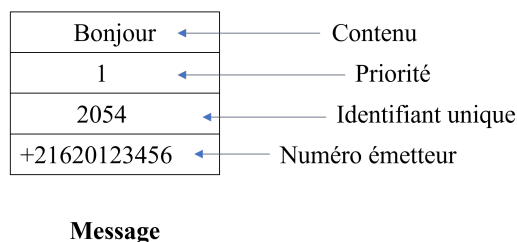


Figure 1

Comme on ne connaît pas le nombre de messages, on optera pour une représentation chaînée de la pile de messages.

1. Donner les structures de données nécessaires pour implémenter la pile en question. (1 pt)
2. Développer l'opération **empiler** permettant d'empiler un message donné dans la pile. (1 pt)
3. Développer l'opération **depiler** permettant de depiler un message de la pile de messages. (1pt)

Dans ce qui suit, lorsque cela est nécessaire, utiliser une seule structure intermédiaire sauf indication contraire dans la question. Les opérations de base sur les Piles vues en classe peuvent être utilisées. Tout autre module utilisé doit être développé.

4. Ecrire un module **Supprimer** permettant de supprimer les n premiers messages d'une pile donnée P . n est un entier positif quelconque. (1 pt)
5. Ecrire un module **SupprimerId** permettant de supprimer d'une pile donnée P le message d'identifiant Id . A la fin de l'opération, la pile P doit contenir tous les autres messages dans le même ordre qu'au début de l'opération. (1.5pt)

6. Ecrire un module `SupprimerSource` permettant de supprimer d'une pile P donnée **tous les** messages reçus d'un émetteur donné `e1`. A la fin de l'opération, la pile P doit contenir les messages des autres émetteurs dans le même ordre qu'au début de l'opération. (1.5pt)
7. Ecrire une fonction `taille` qui étant donnée une pile P, calcule et retourne sa taille. Le contenu de la pile de départ doit rester intacte après exécution. (1.5pt)
8. On désire placer les messages de priorité élevée (priorité 1) **au-dessus** des autres messages dans l'ordre LIFO de départ. En utilisant **exceptionnellement 2 structures intermédiaires**, écrire un module qui étant donnée la pile de messages, permet de l'organiser selon l'ordre indiqué. (1.5pt)

Exercice 2 (10 points)

On souhaite gérer les inscriptions pour l'organisation d'un congrès qui dure deux jours. Un participant peut s'inscrire de manière optionnelle au déjeuner (20 dinars/personne/jour). Il peut aussi de manière optionnelle choisir de passer la nuit dans un hôtel. Il peut choisir un hôtel parmi deux types d'hôtels différents : 3 étoiles (40 dinars/nuit) ou 4 étoiles (70 dinars/nuit). On travaillera avec la structure `Participant` suivante :

`Participant = structure`

`nomPrenom : chaîne`

#on utilisera une seule chaîne de caractères pour le nom et prénom du participant

`dej : entier` *#dej prend la valeur 1 si le participant est inscrit au déjeuner et 0 sinon*

`hotel : entier` *# hotel prend la valeur 0 si le participant ne passe pas la nuit dans un hôtel, sinon il prend 3 ou 4 selon le nombre d'étoiles*

`fin`

L'utilisation de l'opérateur = pour comparer deux chaînes de caractères est possible.

Partie I

On s'intéresse, dans cette partie, à un ensemble de **100 participants**.

1. Définir le type **tableau** `EnsPart` représentant l'ensemble des participants. (0.5 pt)
2. Ecrire une fonction `Montant` qui étant donnée une structure `Participant`, calcule le montant de la facture à payer. (1 pt)
3. Ecrire une fonction **récursive** `NbDej` qui, étant donné le tableau de participants, retourne le nombre de déjeuners à prévoir. (1.5 pt)
4. Ecrire une procédure `Tri_participants` qui étant donné le tableau de participants, le trie de façon à avoir tous les participants qui ne passent pas la nuit à l'hôtel, puis les participants qui ont choisi l'hôtel 3 étoiles puis les participants qui ont choisi l'hôtel 4 étoiles. Les participants ayant fait le même choix d'hôtel doivent apparaître dans l'ordre alphabétique de **nomPrénom**. Utiliser le principe de tri à bulles. (2 pts)

Partie II

Comme en réalité, le nombre de participants n'est pas prévisible, on représentera l'ensemble des participants par une **liste doublement chaînée** de participants.

5. Donner les structures nécessaires à cette représentation (1pt).
6. Ecrire un module `Ajouter` qui étant donnée la liste doublement chaînée de participants permet d'ajouter un participant à la fin. La lecture des informations concernant le participant doit être assurée par le module en question. (2 pts)
7. Ecrire un module `supprimer` qui étant donnée la liste doublement chaînée de participants permet de supprimer un participant de `nomPrenom` donné. (2 pts)

BON TRAVAIL